ECE551: Digital System Design and Synthesis

Fall 2026

Behavioral Verilog

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Behavioral Verilog

- **Structural:** connect existing blocks
- **Dataflow:** specify exactly how each bit is manipulated
- **Behavioral:** describe what the hardware should do as a procedure

- **Goal**: abstract away the details of the hardware implementation to make design easier

- The synthesizer creates a hardware structures that does the same thing as your description
  - … but the synthesizer has to be able to realize your description using real hardware constraints
  - This is why not all Verilog constructs are supported
  - You still need to “think hardware” then describe it using Verilog

Behavioral Verilog

- All behavioral statements occur in procedural blocks: **initial**, **always**
  - Statements are evaluated top to bottom (look a lot like C code)
  - All <LHS> assignments must be to type **reg** or **logic**
  - initial and always blocks cannot be nested
  - Can have multiple blocks, all evaluate concurrently

- **initial** statements start at time 0 and execute once
  - Multiple initial blocks will all start at time 0 and execute independently
  - Not synthesizable, use only in testbench
    - If you are tempted to use initial blocks in your design, it means you are not resetting the flops you need to reset
- **always** statements will evaluate continuously
  - Synthesizable

initial Statement Example

```
`timescale 1 ns / 100 fs

module full_adder_tb;
  reg [3:0] stim;
  wire s, c;

 // instantiate DUT
  full_adder iDUT(sum, carry, stim[2], stim[1], stim[0]);

  // monitor statement is special - only needs to be made once
  initial $monitor("%t: s=%b c=%b stim=%b", $time, s, c, stim[2:0]);

  // tell our simulation when to stop
  initial #50 $stop;

  // stimulus generation
  initial begin
    for (stim = 4'h0; stim < 4'h8; stim = stim + 1) begin
      #5;
    end
  end
endmodule
```

All initial blocks start at time 0

Not a very good example…

- Do we need 3 initial blocks here?
- Are these begin/end pairs necessary?

initial Statement Example

```
module stim();

  reg m,a,b,x,y;

  initial
    m = 1'b0;

  initial begin
    #5  a = 1'b1;
    #25 b = 1'b0;
  end

  initial begin
    #10 x = 1'b0;
    #25 y = 1'b1;
  end

  initial
    #50 $finish;

endmodule
```

What events at what times will a verilog simulator produce?



Time	Event
0	m = 1'b0
5	a = 1'b1
10	x = 1'b0
30	b = 1'b0
35	y = 1'b1
50	$finish

always Statements

- Operates continuously (always)
  - Executes at time 0, but loops continuously
  - Can use a trigger list to control operation (sensitivity list)
    - @(a,b,c)
    - In absence of a trigger list, it will re-evaluate when the last <LHS> assignment completes

```
module clock_gen (output reg clock);

  initial
    clock = 1'b0;          // must initialize in initial block

  always                   // no trigger list for this always
    #10 clock = ~clock;    // always will re-evaluate when
                           // last <LHS> assignment completes
endmodule
```

Sensitivity Lists

- Conditionally evaluate inside of **always** block
  - Any change on sensitivity list triggers block

```
always @(a,b,c) begin
  ...
end
```

- Original way to specify trigger list: `always @(a or b or c)`
- In Verilog 2001 can use **,** instead of **or**: `always @(a, b, c)`
- Verilog 2001 also has * for combinational only: `always @(*)`
  - Anything on the RHS of assignments are included in the list

```
always @(*)
  if (a==b) c = d ^ e;
  else c = d;
```



Sensitivity list is @(a,b,d, e)

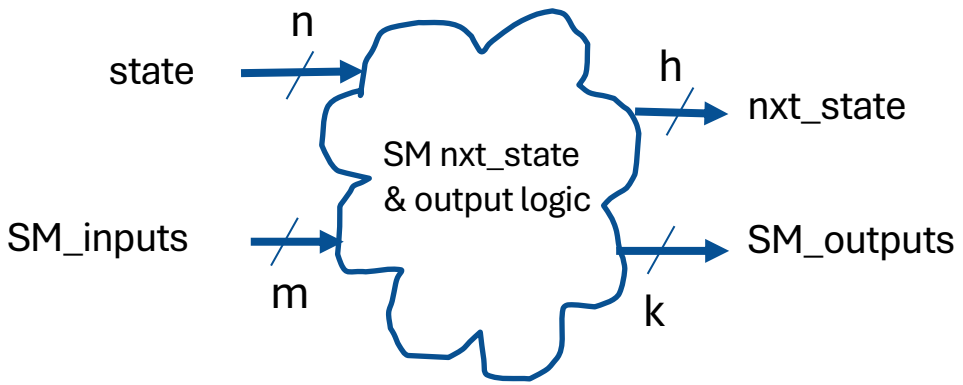
always_comb

- System Verilog introduced **always_comb**
  - Does not have a sensitivity list
  - More descriptive than always@(*)
  - Simulator will warn if you did not infer combinational logic with it

```
////////////////////////////////////////////////
// Infer state transition logic & SM outputs //
////////////////////////////////////////////////
always_comb begin
  ////////////////////////////////////////
  // Default outputs and nxt_state logic //
  ////////////////////////////////////////
  nxt_state = state;
  capture_cmd = 0;
  nxt_cmd = 0;
  go = 1;

  case (state)
    STOP: begin
      go = 0;              // only state we are not going
      if (cmd_rdy & line_present) begin
      // if (line_present) begin
        capture_cmd = 1;
        nxt_state = GOING;
      end
    end
    GOING : begin
      if (!line_present) begin
```

SM state transition logic & output logic are purely combinational

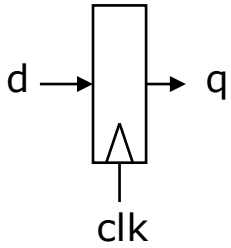


Flip Flop Inference

- Output of flip flops change at the rising (or falling) edge of clock
- Described using always block
- In sensitivity list, specify **edge-sensitive** to clock (vs level-sensitive)
  - Block evaluated when there is an edge (vs when signal value changes)
  - Use keywords **posedge** and **negedge**
- A **posedge** is on the transitions
  - 0 → x, z, 1
  - x, z →1
- A **negedge** is on the transitions
  - 1 → x, z, 0
  - x, z → 0

```
reg q;

always @(posedge clk)
  q <= d;
```

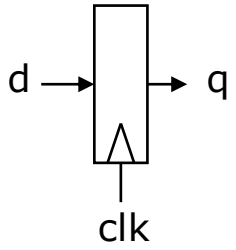


Flip Flop Inference

- D-FF with no reset

```
reg q;

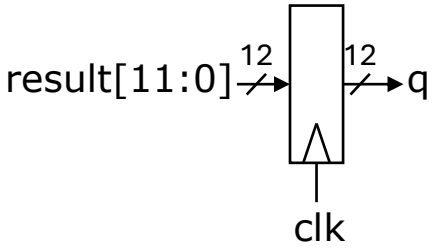
always @(posedge clk)
  q <= d;
```



- Infer multiple D-FFs with no reset

```
reg [11:0] q;

always @(posedge clk)
  q <= result[11:0];
```



Be careful… Yes, a non–reset flop is smaller than a reset Flop, but most of the time you need to reset your flops.

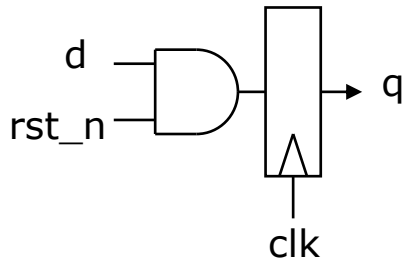
Always err on the side of resetting the flop if you are at all uncertain.

Flip Flop Inference

- D-FF with synchronous reset

```
reg q;

always @(posedge clk)
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
```

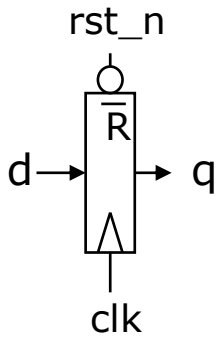


Many cell libraries don't contain synchronous reset flops. This means the synthesizer will have to combine 2 (or more) standard cell to achieve the desired function.

- D-FF with asynchronous reset

```
reg q;

always @(posedge clk, negedge rst_n)
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
```



Cell libraries will contain an asynch reset flop. It is usually only slightly larger than a flop with no reset. This is probably your best bet for most flops.

Reset has its affect asynchronous from clock. What if reset is deasserting at the same time as a +ve clock edge? Is this the cause of a potential metastability issue?

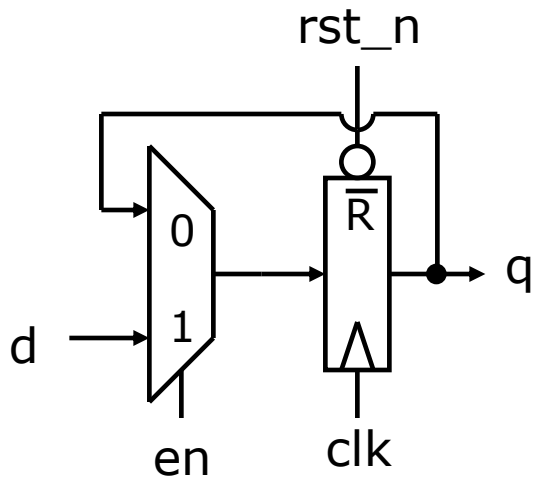
Flip Flop Inference

- Conditionally enabled Flops

```
reg q;

always @(posedge clk, negedge rst_n)
  if (!rst_n)
    q <= 1'b0;   //asynch reset
  else if (en)
    q <= d;      //conditionally enabled
  else
    q <= q;      //keep old value
```

omit this since it is redundant
reg type maintains old value by default



Flip Flop Inference in System Verilog

- System Verilog introduced **always_ff**
  - Tells the tool you intend to infer a flip flop
  - Will get warning if it did not infer a flop

```
module simple_ff(input clk,d,rst_n,
                 output q);

reg q;

always @(posedge clk, negedge rst_n)
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
endmodule
```

Standard Verilog

```
module simple_ff(input clk,d,rst_n,
                 output q);

logic q;

always_ff @(posedge clk, negedge rst_n)
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
endmodule
```

System Verilog